University of Deusto

Bachelor in Data Science and Artificial Intelligence

INTEGRATION OF VISION LANGUAGE MODELS (VLMS) IN A MULTI-AGENT REINFORCEMENT LEARNING (MARL) ENVIRONMENT FOR COOPERATIVE ROBOTIC TASKS IN NVIDIA ISAAC SIM

Student Unai Olaizola Osa

DNI 73032586L

Email unai.olaizola.osa@opendeusto.es

GitHub repository VLMs IsaacSim

April 17, 2026

Acknowledgments

Abstract

This project proposes the development and implementation of a Multi-Agent Reinforcement Learning (MARL) system integrated with Vision Language Models (VLMs) for the execution of cooperative robotic tasks within a simulated NVIDIA Isaac Sim environment.

One of the major challenges lies in translating commands from a natural language format into precise robotic actions. To address this, the reasoning capabilities of the VLMs have been exploited for complex instruction interpretation and decision-making. A controlled environment where robotic agents must cooperate to manipulate and respond to objects in order to achieve a shared goal has been designed and shaped for Reinforcement Learning training using NVIDIA's Isaac Lab framework. The integration of these models has enhanced generalization and promote a more cooperative behavior of the agents against variations in the scene. The learning efficiency has been evaluated in addition to metrics such as the stability of the policies and the transferable capacity of the model(s). Additionally, the project includes an in-depth analysis of state-of-the-art (SOTA) models, providing a comprehensive review of the current methodologies and core concepts that form the theoretical foundation of the proposed architecture, presented in the documentation.

The expected outcome is a demonstration and reflection of the empowerment of multi-modal models within robotic environments, which can be applied to real-world problems.

Index terms

Vision language models, multi-agent reinforcement learning system, computer vision

Contents

List of Figures

List of Pseudocodes

1. INTRODUCTION

1.1 PROBLEM CONTEXT

1.2 MOTIVATION

1.3 MAIN OBJECTIVES AND SPECIFIC OBJECTIVES

1.4 PROJECT SCOPE

2. STATE OF THE ART

2.1 EVOLUTION OF MULTIMODAL LEARNING

Multimodal learning is a type of *deep learning* that processes multiple types of data referred as modalities [1]. That data may belong to different domain such as text, audio, images or video. Multimodal learning was proposed at the beginning of the DL period, around year 2011, whilst have raised their popularity when they were mixed with large multimodal languages in the recent years.

2.1.1 Multimodal deep Boltzmann machines

Boltzmann machines are known to be the multimodal data dealing algorithm ancestors. Invented in 1985 and named after the *Boltzmann distribution*, these machines can be seen as stochastic neural networks. Multimodal deep Boltzmann machines can process and learn from different types of information simultaneously, having a separate machine for each modality. Its units are divided into visible and hidden units, units which represent neurons with a binary output indicating whether they are activated or not.

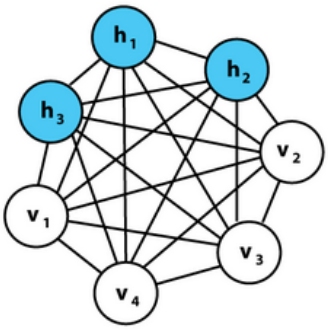

Figure 2.1: Deep Boltzmann machine.

However, learning is impractical using these machines because of the exponential computational time and not optimized architecture. Successor *restricted Boltzmann machine* alternative was designed to deal with efficiency by just allowing connections between hidden units and visible units. Lowering the computational power and making training more efficient.

2.1.2 Multimodal Transformers

Transformers can also be adapter for modalities, usually finding a way to *tokenize* the data domain. Even if multimodal transformers can be trained from scratch, a 2022 *transfer learning* study showed that fine-tuned transformers originally pretrained with natural language data can show competitiveness in logical and visual tasks.

2.1.2.1 The Perceiver

Perceivers, are a variant of the transformer architecture exclusively adapted for processing multimodal data. The Perceiver is designed as a general architecture that can learn simultaneously from large amounts of heterogeneous data, implementing *asymmetric attention* mechanisms. Asymmetric attention consists of architectural modifications that break the "standard symmetry" to address specific domain constraints. This approach leads to outperforming specialized models on classification tasks. It iteratively alternates cross-attention and latent self-attention blocks to project a high-dimensional input byte array to a fixed-dimensional latent bottleneck before processing it using a deep stack of transformer-style self-attention blocks in the latent space [4].

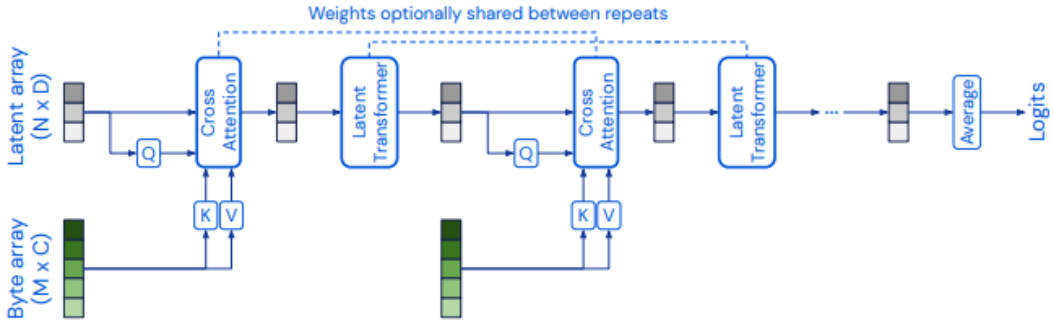

Figure 2.2: Perceiver architecture from the referenced paper

2.1.2.2 Image generation Transformer models

- **DALL-E 1** (2021): When it comes to image generation models, *DALL-E 1* is not a *difussion model* (latent variable generative models). It uses a decoder-only transformer which autoregressively generates a text followed by the token representation of an image, which is ten converted by a variational autoencoder to an image.

- **Parti** (2022): On the other hand, models like *Parti* use an encoder-decoder schema. Where the encoder processes a text prompt and the decoder generates the token representation of an image.

- **Muse** (2023): uses an encoder-only architecture trained to predict the masked image tokens from unmasked image tokens. In training, all input tokens are masked, and the highest confidence predictions are revealed

- **Phenaki** (2023): is a text-to-video-model which uses a bidirectional masked transformer conditioned on pre-computed text tokens. Then, the tokens are decoded to a video.

2.1.3 Applications of Multimodal Learning

The integration of data from different domains has allowed a deeper understanding of models for different type of tasks requiring multimodality:

- **Cross-modal retrieval**: is a subfield of information retrieval which enables users to search and retrieve information across different data modalities [12]. Differing from the traditional retrieval systems, cross-modal retrieval bridges different types of media to facilitate flexible information access. There are several combinations of retrieval depending on both the input and output data domain, e.g. *audio-to-video, video-to-text, ...*

- **Content generation**: models such as *DALL-E* generate images from textual descriptions.

- **Classification and missing data retrieval**: multimodal *Deep Boltzmann Machines* outperform traditional shallow machine learning algorithms like the *support vector machines* in classification tasks and are able to predict missing instances in multimodal datasets.

- **Robotics and human-computer interaction**: multimodal learning has improved the interaction in robotics and AI by integrating sensory inputs like speech, vision and touch, aiding autonomous systems and human-computer interaction.

2.2 ADVANCEMENTS OF VISION LANGUAGE MODELS

While the multimodal models mentioned before established the standard workflow for processing heterogeneous data, they often treated modalities independently to be combined or translated. The next step in the evolution are the *Vision Language Models*, representing a shift towards a more reasoning approach

rather than bare multimodal output goals. Instead of just generating an image from text or retrieving text from a video, VLMs think across all the domains simultaneously, implementing the reasoning capabilities from *large language models* to interpret visual scenes.

VLMs are the result of the evolution to earlier image captioning systems, systems which were designed primarily to take isolated images and produce descriptions. However, this systems just received the image as input, not an image-text pair input as the vision language models do now [17]. I will now go over some of the huge advancements that resulted when transitioning from the traditional multimodal learning to these new vision language models and their main differences:

- **Different input handling**: while with multimodal learning you would have a separate model for handling data from a specific domain e.g. one for dealing with text and another for video, VLMs were designed to receive input pairs of data from different modalities. Usually being (image, text) pairs. The VLM is able to align concepts belonging to different modalities inside a single shared vector space.

- **Change of objective**: multimodal learning mainly covered traduction problems from one domain to another, whereas modern VLMs seek inter-domain reasoning. The capability to reason and distinguish the same concept in different modalities.

- **Representation in the latent space**: vision language models try to preserve the same mathematic meaning for the same concept across all domains.

2.2.1 Earliest models

The early image captioning systems from around year 2010 used the *encoder-decoder* architecture, which will be described later on. These models would encode the patched images and vectorize them into feature vectors, which were then fed to the decoder part in order to generate the associated output description. When it comes to the strategies implemented by this earliest models include combination of handcrafted *visual features* to encode the image patches and *n-gram* templates to generate the description.

Those visual features can refer to any visual property like points, edges or even objects in the image.

On the other hand, n-grams are statistical language models that calculate the probability of the next word in a sequence from a fixed size window of previous words. Depending on the number of previous words considered, we can get bigram models if just one previous word is considered, if it's two is a trigram

model, ... up to n - 1 considered as *n-gram* models and finally an unigram if no previous context is considered.

The general approximation for a sequence of words $w_1, \ldots, w_m$ is:

$$P(w_1, \ldots, w_m) \approx \prod_{i=1}^{m} P(w_i \mid w_{i-(n-1)}, \ldots, w_{i-1})$$

- Unigram Model ($n = 1$): No context, independent probabilities.

$$P(w_1, \ldots, w_m) \approx \prod_{i=1}^{m} P(w_i)$$

- Bigram Model ($n = 2$): Depends on the single previous word.

$$P(w_1, \ldots, w_m) \approx \prod_{i=1}^{m} P(w_i \mid w_{i-1})$$

- Trigram Model ($n = 3$): Depends on the two previous words.

$$P(w_1, \ldots, w_m) \approx \prod_{i=1}^{m} P(w_i \mid w_{i-2}, w_{i-1})$$

2.2.2 Deep learning based models

When the deep learning neural networks arose and became dominant near year 2015, newer models and strategies also emerged. Convolutional neural networks (CNNs) started to being used for image encoding and recurrent neural networks (RNNs) to generate the captions.

Despite their respective success in independent tasks, the combination of both models hit a plateau when dealing with VLM scenarios due to their infrastructure limitations. Convolutional neural networks work great when capturing local spatial patterns but they struggle with global long-term dependencies of the whole image without a large number of layers. On the other hand these *seq2seq* models performed poorly with long-term dependencies due to their hard-coded sequentiality. Often leading to the model forgetting the processes carried in the first layers of the architecture.

2.2.3 The Transformers architecture Standard

The transition from sequential models combined with convolutional neural networks to *Transformers* solved the long-term dependency capturing issues and lack of global visual feature captioning. Still using these models, VLMs are able to attend to the whole input prompt simultaneously thanks to parallelization and attention mechanisms and also due to the specific vision transformer architectures.

The functionality of the basic Transformers architecture can be wrapped into three main stages:

1. The encoders transform the input sequence into high-dimensional numerical representations called *embeddings* which capture all the semantic and positional meaning of the tokens belonging to the input. In order to deal with those high-dimensional embeddings in the next steps of the architecture where the order of the input sentence words cannot be distinguished as the transformer processes all tokens in parallel, the *positional encoding* was implemented. These encodings typically use sine and cosine functions of different frequencies to provide a unique signature for each position:

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

   This ensures that the model captures the structural context (e.g., distinguishing between "dog bites man" and "man bites dog") before any attention is calculated.

2. Then they use a mechanism called *self-attention* which allows the encoder to focus on the most relevant tokens of the sequence independent of their position. This is where the magic of the transformers actually happens. The implementation of the attention mechanism replaced the use of previous *seq2seq* models which relied on recurrence. The original paper also introduced the concept of *scaled dot-product attention*:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

   where respectively Q, K, V are the query, key and value matrices, and d_k the dimensionality value. This approach eliminates the need for RNNs, ensuring parallelizability for the architecture. In the self-attention mechanism, those matrices are dynamically generated for each input sequence, allowing the model to focus on the different parts of the input sequence at different timesteps. *Multi-*

head attention enhances the process by introducing several parallel attention heads simultaneously, enabling each head to learn different Q, K and V matrix linear projections and focusing on multiple aspects instead of a single one. MHA ensures that the input embeddings are updated from a more varied and diverse set of perspectives, and after the attention outputs are calculated, they are concatenated and passed through a linear layer transformation to generate the output.

3. Finally the decoders of the transformers take both the output of the self-attention mechanism and the encoder generated embeddings to generate the most statistically likely output sequence. In the case of VLMs, it can also use those attention-weighted embeddings to output the most loyal visual representation compression.

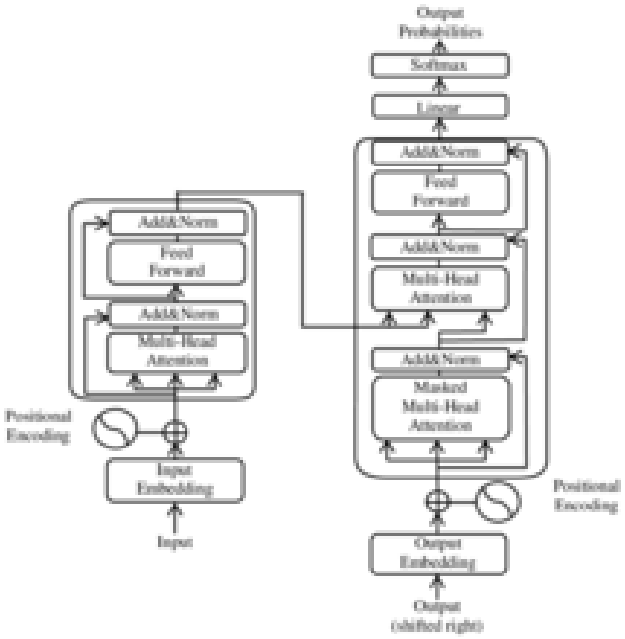

Figure 2.3: Original Transformers architecture published in *"Attention is all you need"*

2.2.3.1 VLM tasks with Transformers

The transformer architecture replaced the sequential recurrent neural networks in the role as decoders. In parallel, the network parameter training started relying on image-text pair datasets, and the scope of VLMs widened until reaching other tasks.

- **Phrase grounding**: Among those tasks, *phrase grounding* could be mentioned. Task involving

both natural language processing and computer vision where words or phrases within a sentence are associated to corresponding patches in an image [9]. Consists of identifying the spatial location within an image that aligns with the meaning of a given phrase, enabling models to map visual and textual data.

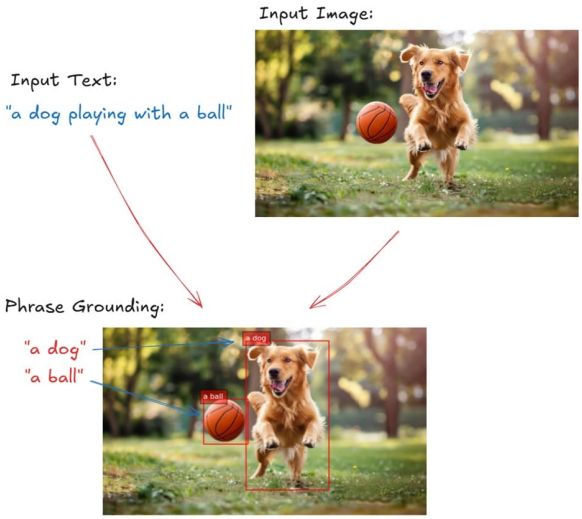

Figure 2.4: Graphical example of phrase grounding.

- **Visual Question Answering** (VQA): Another task which still remains as perhaps the most relevant one is *visual question answering*. Specific task where the model is provided with an image and a specific related question, and it must generate an accurate response to it based on the visual content. Being its core pillars the object detection duty for identifying the objects in the image, then scene understanding to map the relations between objects and then finally text comprehension for embedded text interpretation within an image [3].

However, modern VLMs do not follow the conventional 2017 paper [10] workflow, they typically use different transformers configuration. Some models use only the encoder part of the architecture for both modalities, completely ignoring the traditional encoder component. While other recent models use the encoder for feature extraction and use an independent encoder for language modeling. Different approaches that will be explained in the following sections.

2.3 FROM ENCODERS TO GENERATIVE LLMS

2.3.1 Language encoder

In traditional multimodal systems, the language encoder functioned as a bidirectional processor, having the role of generating contextualized latent representation of the input where each token could attend to its surrounding tokens in both directions. However, in modern VLM architectures, the encoder has been linked to the LLM engine. Functioning as a preparation of the linguistic information to interact with the visual tokens within a shared vectorial space.

2.3.2 The LLM transition: from encoders to decoder-only architectures

The most significant paradigm shift in VLM evolution is the transition from symmetric *encoder-decoder* systems to *decoder-only* architectures. This move toward generative LLMs is driven by three fundamental technical advantages:

- **Causal Attention vs. Bidirectional Attention**: Unlike encoders that allow tokens to see the entire sequence at once, decoders implement a *causal mask*. This ensures that during training and inference, each token can only attend to itself and preceding tokens. This directional dependency is what enables autoregressive generation, the ability to reason and predict the next word in a conversation.The causal mask M is typically a lower triangular matrix, where the attention score between position i and j is set to $-\infty$ if $j > i$:

$$M_{ij} = \begin{cases} 0 & \text{if } i \geq j \\ -\infty & \text{if } i < j \end{cases}$$

  After applying the *softmax* activation function, these $-\infty$ values become zero probability, effectively blocking information flow from future tokens.

- **In-Context Learning and Reasoning**: By utilizing a decoder-only system, the VLM treats the input image as a visual prefix. The model does not just classify the image, rather it predicts the logical continuation of a sequence $[Image, Instruction, Response]$. This allows the model to influence *Chain-of-Thought* (CoT) reasoning, where it can articulate its thought process before arriving at a final answer.

- **Scalability and Unified Training**: Decoder-only models are simpler to scale because they consolidate the entire reasoning process into a single set of weights. In a model like *LLaVA*, the LLM receives a unified sequence of tokens: $\mathcal{S} = [v_1, \ldots, v_n, t_1, \ldots, t_m]$, where v represents aligned visual pseudo-tokens and t represents text tokens. The model treats these two modalities as a single language, significantly reducing the architectural bottleneck between "seeing" and "speaking."

2.4 VISION TRANSFORMERS

2.4.1 Vision encoder & Vision Transformers (ViT)

Compared to earlier VLMs which used *convolutional neural networks* (CNNs) for feature extraction such as the colors, shapes and textures from the input, modern ones employ an architecture called *visual transformer* (ViT).

Visual transformers process images into smaller patches (rather than text tokens) and treats them as sequences, implementing also the self-attention mechanism across the patches to create a transformer-based representation of the input message.

ViTs were designed as *convolutional neural network* alternatives in computer vision applications, having different biases, training stability and data efficiency. Compared to CNNs, vision transformers are less efficient yet they have higher capacity.

The image available below 2.5 references the original architecture published in the 2020 paper [2], being basically a *BERT*-like encoder-only transformer. It also uses special token `<CLS>` in the input side to refer to the start of the sentence token, and the output vector is used as the input of the final MLP head. Transformers, including the ViTs, measure the relationship between pairs of input tokens with the attention mechanism. In the case of images, the basic unit of analysis is the pixel. Vision transformers compute relationships between small regions of pixels (patches) and they maintain the spatial order with the positional embeddings computation. Each section of the architecture is passed through a linear sequence and multiplied by the embedding matrix, being the result with the position embedding fed to the transformer. Let's overview its functionality:

1. The input image is of type $\mathbb{R}^{H \times W \times C}$, where H, W, C are height, width, and channel (RGB). It is then split into square-shaped patches of type $\mathbb{R}^{P \times P \times C}$. For each of the patches, the respective "patch embedding" is obtained by pushing the patch through a linear operator and transformed

through the *positional encoding* layer. Both embeddings are added and pushed to the various transformer encoder layers.

2. Inside those encoder layers, the attention mechanism incorporates more and more semantic relations to the image patch embeddings.

3. Depending on the end task desired, one could add different layers in the end steps. For instance, for a classification task like the one performed in the image, a shallow *multi-layer perceptron* (MLP) layer is added on top to output the probability distribution over the classes. As fact, the original paper goes for a linear-GeLU linear softmax network for classification.

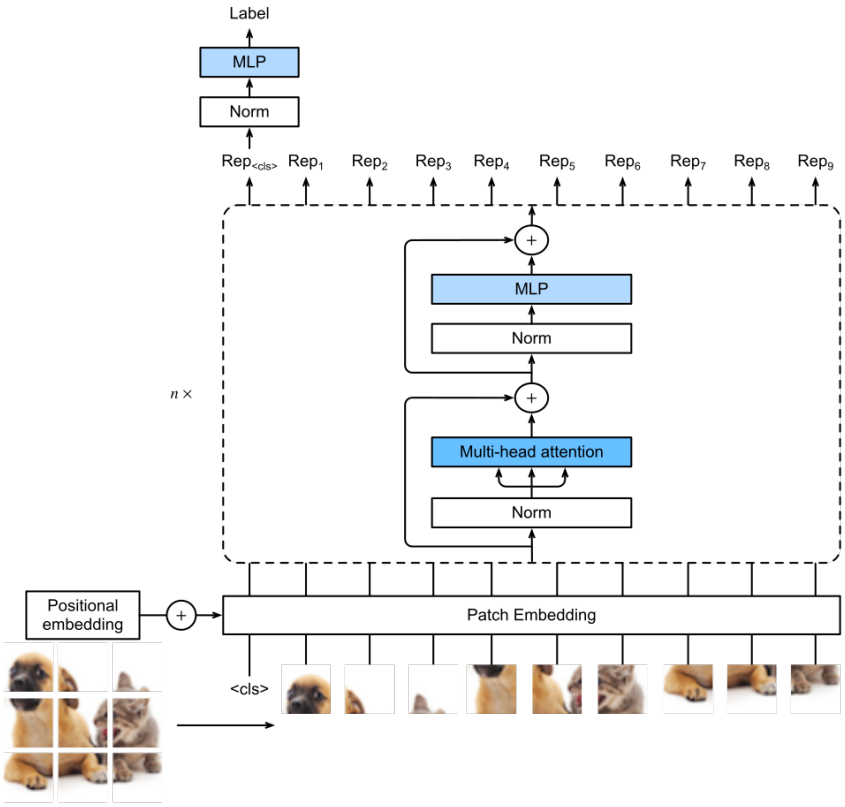

Figure 2.5: Vision transformer architecture

There are other ViT variants which have been published and reported having used different techniques to deal with image manipulation. The following are some of the architectural improvements:

- **Global average pooling (GAP)**: unlike the original paper where the `[CLS]` token is used to

represent the beginning of the sequence emulating BERT, this approach does not use it. It simply takes the average of all output tokens as the final classification token. In the original report, it was mentioned to be equally efficient to the original approach.

- **Multi-head attention pooling (MAP)**: it applies a new *multi-head attention* block to pooling. It takes the output of the ViT this being the list of vectors $x_1, x_2, \ldots, x_n$, whereas its output is MultiheadedAttention(Q, V, V). Where Q is the trainable query vector and V the matrix with the rows being the input vectors. More recent papers state that both GAP and MAP show a better performance than the basic BERT-like pooling.

- **Masked Autoencoder**: this architecture involves two vision transformers put end-to-end. The first one takes the input image patches with positional encoding and output the vectors representing each patch (same encoder procedure seen until now). This is where the second component being the decoder (even if it still an encoder-only transformer) takes the first encoder's output with positional encoding too and outputs the image patches again. See the architecture in the image below 2.6.

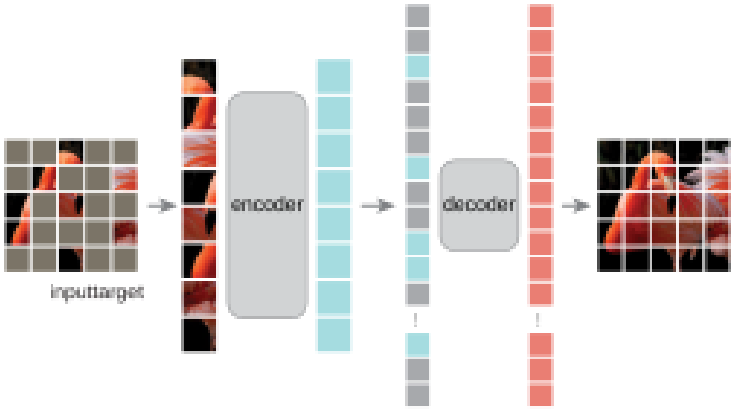

Figure 2.6: Masked autoencoder architecture

During the training process, a percentage of the splitted patches is masked by special mask tokens, while leaving others untouched (the sweet spot has been said is around 75% mask ratio). The whole network's task is to reconstruct the whole image with the missing "puzzle pieces" (being the masked patches). Masked patches are added back to the output of the encoder block as mask tokens and both are fed into the "decoder". A reconstruction loss is computed for the masked patches to assess network performance. In the prediction stage, the decoder is left over, the input image patches are processed and the resulting vector representation is fed to the encoder.

- **DINO (self-DIstillation with NO labels)**: like the masked autoencoder procedure, DINO is a self-supervised way to train a vision transformer. Acting as a "teacher-student" method, the student being the model itself and the teacher, an exponential average of the student's past states, must be able to transfer its knowledge to the smaller model [15]. The loss function it uses is the *cross-entropy loss* between the output of the teacher network and the output of the student version. The inputs of the networks are two different crops of the same image, represented as $T(x)$ and $T'(x)$, where x is the original image. And as mentioned before, the network of the teacher represents the exponential decay average of the student's past parameters:

$$\theta_{t'} = \alpha\theta_t + \alpha(1-\alpha)\theta_{t-1} + \dots \tag{2.1}$$

  All put together, the loss function stands as followed:

$$\mathcal{L}(f_{\theta_{t'}}(T(x)), f_{\theta_t}(T'(x))) \tag{2.2}$$

In order to finish the ViT research subsection, I would like to make a little comparison with their "ancestor" model, the *convolutional neural networks* (CNNs).

The fundamental distinction between Vision Transformers and the convolutional neural networks lies in their inductive biases. CNNs are built on the principles of locality and translation invariance, using sliding kernels that assume pixels close to each other are more related than those far apart. This bias allows CNNs to be highly efficient with smaller datasets and provides a natural robustness to local perturbations and noise. In contrast, ViTs discard these spatial assumptions. By treating an image as a sequence of patches and applying a global self-attention mechanism, a ViT can capture long-range dependencies between distant parts of an image from the very first layer. This flexibility allows ViTs to surpass CNN performance on massive datasets where the model can "learn" the spatial relationships from scratch, rather than having them hard-coded into the architecture.

However, this architectural freedom comes with a significant trade-off in terms of optimization and data requirements. Because ViTs lack the "shortcuts" provided by the convolutional structure, they are notoriously sensitive to hyperparameter choices and the selection of optimizers, often requiring advanced strategie and specific learning rate schedules to achieve stability. While a CNN's computational complexity scales linearly with the number of pixels, the self-attention in ViTs is quadratically complex $O(N^2)$ relative to the number of patches N, making high-resolution processing more resource-intensive. Con-

sequently, while CNNs remain the more time-efficient and stable choice for many standard applications, ViTs have become the preferred backbone for large-scale Vision-Language Models due to their superior ability to scale and their capacity for modeling the complex, global context required for multimodal understanding.

2.5 MULTIMODAL ALIGNMENT: CLIP

2.5.1 CLIP: Contrastive Language-Image Pretraining

In the year 2021, OpenAI launched the approach that would totally revolution the evolution of vision language models, they released the method of *contrastive language-image pretraining* also known as *CLIP*. Technique for training an image-text pair into neural network models, having an exclusive neural network for image and text understanding purposes, adopting a contrastive objective.

When it comes to the algorithm itself, each neural network receives the input in the corresponding format and returns a single semantic/visual representative vector [14]. The objective to maximize by both models is the best alignment between both output vectors in the shared vector space, mapping the similar text-image pairs as close as possible while putting apart the most dissimilar ones. The training relies on a huge dataset with image-text pairs as and the two output vectors are considered similar depending on the *dot product* value.

$$s_{i,j} = \mathbf{v}_i \cdot \mathbf{w}_j = \sum_{k=1}^{d} v_{i,k} w_{j,k} \tag{2.3}$$

where $\mathbf{v}_i$ and $\mathbf{w}_j$ represent the embedding vectors generated by the text and image encoders respectively, and d denotes the dimensionality of the vector space. The loss function used is the multi-class N-pair loss, which is a symmetric *cross-entropy loss* over the similarity scores. The function fosters the dot product between the matching image and text vectors to be as high as possible, while discouraging high dot products between mismatching pairs.

$$\mathcal{L} = -\frac{1}{N} \sum_{i} \ln \frac{e^{v_i \cdot w_i / \tau}}{\sum_j e^{v_i \cdot w_j / \tau}} - \frac{1}{N} \sum_{j} \ln \frac{e^{v_j \cdot w_j / \tau}}{\sum_i e^{v_i \cdot w_j / \tau}} \tag{2.4}$$

being parameter T the temperature parameter ($T>0$) which is parametrized and learned during the training. Other loss functions are possible, such as the *Sigmoid CLIP* function.

Once gone over the algorithm's properties, let's dive deep into the architecture:

- Image model: when it comes to the image encoder used in most of the CLIP versions, they all usually are some kind of *vision transformers* (ViT). Vision transformers often vary on the patch size, being that size the nxn dimensions of the patch the image was divided into. Alternatively, *convolutional neural networks* (CNNs) can be used as well (such as *ResNet*), the essential thing is to ensure the sizes of the image and text model output vectors are the same size.

- Text model: the text encoding models are usually *transformers*. Even if it is heavily inspired by the transformers report published by OpenAI [5] and shares similarities with the *GPT* architecture (decoder-only), the text model serves as a text encoder. Furthermore, its functionality is inspired by encoder only models such as *BERT*, while sharing the dimensionality and number of parameters with decoder-only models as mentioned before.

After going over the properties of both encoders that make the whole CLIP model, I will detail the model's workflow inspired on the image below 2.7.

1. The image of the image-text pair input is first splitted into smaller sized images named patches and then fed into the vision encoder, which as mentioned earlier, can be any kind of architecture such as a vision transformer (ViT) or convolutional neural network (CNN). The output of the vision encoder will in fact be a vector representation of all the features that have been captured: shapes, texture, colors, objects, spatial relations, ... etc.

2. Now it's time the model turns those embeddings into real semantic representations, nevertheless, the model still does not know how to label concepts directly. So the model must know combine the embeddings obtained in the image model with the ones from the text model output, aligning them into a shared vector space establishing the real relations and links between the pairs. This bridging mechanism between the image and text pair (which will be described deeper in the corresponding section) is where the CLIP model's magic really shines. CLIP uses a contrastive alignment strategy in order to put correct pairs closer and dissimilar pairs further in the vector space.

3. In the standard CLIP workflow, the model does not generate new text; instead, it performs *zero-shot prediction* by calculating the similarity between the image embedding and a set of candidate text embeddings. It identifies which text snippet has the highest dot product with the image, effectively "selecting" the best label.

4. In more advanced generative extensions, these aligned visual features are passed to an LLM. The LLM treats the image embeddings as part of its input sequence, allowing it to generate complex descriptions or provide answers to specific questions in a task known as *Visual Question Answering* (VQA).

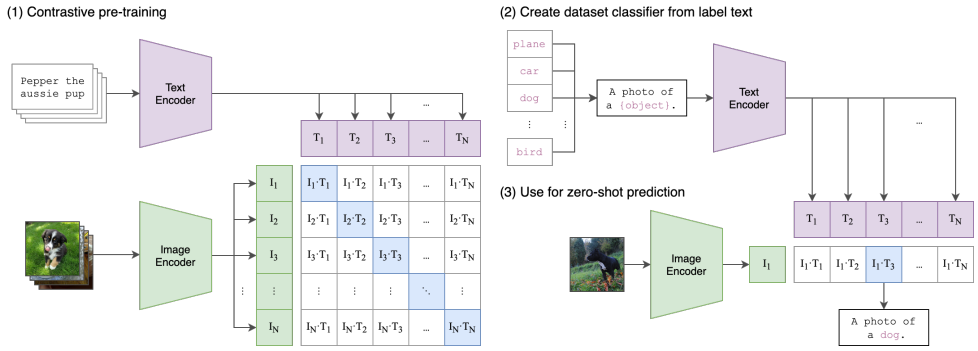

Figure 2.7: CLIP workflow

The reason why these models work so well is because of the millions of image-text pair instances used for training and due to the model's ability to learn and deal with statistical spatial correlations that are shown both in the images and texts.

The dataset the model was trained on is a *WebImageText* (WIT) containing more than 400 million pair instances scraped from the internet, almost half a million text queries and more than 20,000 image-text pairs as query. All of those queries were generated by the most frequent words of the english Wikipedia and then extended using bigrams. The dataset remains private and has never been released.

Last but not least, images are preprocessed by dividing them into the RGB (reed, green and blue) channels and normalizing the values so they fall between range 0 to 1. Finally those values are merged with the mean and standard deviation of the images from the training dataset. In case the input image does not have the desired resolution (224x224 usually) the image is scaled by *bicubic interpolation* [13] to the shorter sides is the same as the native resolution, and then the central square of the image is cropped out.

2.6 MODALITY BRIDGING AND PSEUDO-TOKENS

2.6.1 Modality bridging mechanism

The bridge briefly mentioned in previous sections, serves as the "mathematical handshake" between the visual and linguistic domains. Because the vision encoder and the LLM are often trained independently, their output dimensions and feature spaces rarely match. It's in those scenarios where the *dimensionality alignment* technique is applied. For example, an encoder such as ViT-L/14 may produce embeddings in a 1024-dimensional space (1024 embedding size), whereas a Llama-based LLM requires 4096-dimensional inputs.

To deal with different size embeddings, several algorithms have been developed, being the *Manifold alignment* [16] one of them. This algorithm assumes that both the image and textual data sets exist in different high-dimensional spaces, still they share an underlying common semantic structure, also called *manifold*. The bridge's true purpose is to find a mapping that preserves the local geometric relationships of the two disparate sets. By minimizing the distance between related points while preserving the intrinsic structure of each domain. A great result would indicate the ability of the model to surpass the modality gap and project the same token close to each other across all modality spaces.

2.6.2 Pseudo-Tokens

Once the different embeddings are aligned, tokens are referred as *pseudo-tokens* or *visual tokens*. There is an interesting paper where this kind of tokens are analyzed more in depth, I am talking about the *A Sentence is Worth 128 Pseudo Tokens: A Semantic-Aware Contrastive Learning Framework for Sentence Embeddings* paper [7]. The study argues that a single sentence embedding is still not enough to capture visual complex meaning, it states that is necessary to use a fixed number of these new generated pseudo-tokens to actually represent a concept or image. In the context of VLMs, pseudo-tokens ensure that the LLM is able to look for different parts of the visual manifold as if they were a structure sequence of words, making easier to align between high-dimensional visual and semantic features.

When it comes to practical cases, this alignment is implemented through varying levels of architectural complexity. The simplest and most common approach, seen in models like LLaVA, utilizes a *linear projection* or a shallow *multi-layer perceptron* (MLP) to perform the manifold mapping. On the other hand, more advanced architectures like Flamingo or BLIP-2 use bottleneck structures such as the *perceiver resampler* or the *q-former*. These use a set of learned queries to attend to the encoder's output, effectively

summarizing the visual space into a fixed, manageable number of pseudo-tokens. This prevents the LLM from being overwhelmed by the high-resolution raw output of the vision encoder, especially when dealing with high-resolution images or video sequences.

2.7 TRAINING STRATEGIES IN VLMS

Vision language models are trained not only in a single step process, but in a multi-stage training workflow, starting from the alignment of two disparate data modalities going to refining the model's ability to deal with natural instructions. The strategies can be categorized into three paradigms:

2.7.1 Contrastive Pre-Training

: The core of contrastive methods is the idea of learning how to contrast between pairs of similar and dissimilar data points [6]. A pair of similar data points is called positive sample if both points are different representations of the same data instance, in our case multimodal representations of the same instance. Whereas the negative samples are those pairs in which the data points belong to different examples. While computer vision methods learn from input-input (image-image) pairs and natural language processing methods deal with input-output (text, label) pairs, vision language models receive input-input (text, image) pairs. Previously mentioned models such as *CLIP* [5], apply this learning approach to maximize the *cosine similarity* distance metric between matching image-text pairs in a shared embedding space, aiming to maximize the metric for the positive samples while minimizing the negative ones. The loss function used is the *cross-entropy* function.

2.7.2 Generative Pre-Training

While contrastive learning focuses on aligning data points from different modalities in a shared single embedding space, generative pre-training shapes that task as a sequence-to-sequence problem. Using thie *Prefix Language Modeling* (PrefixLM) approach [11], the model receives a prefix consisting of a pair of image patches and a text sequence. The goal of the model is to predict the continuation of that textual sequence, just like *seq2seq* models like *RNNs* and *LSTMs* try to predict. The loss function used is the negative log-likelihood loss, identical to the ones used in large language models like *GPT*:

$$\mathcal{L}_{PrefixLM} = -\sum_{t=1}^{|y|} \log P(y_t \mid y_{<t}, x_{prefix})$$

where x_{prefix} represents the combined image and text prefix and y_t is the text token at time t.

2.7.3 Visual Instruction Tuning

: This strategy involves fine tuning the model with smaller datasets of higher quality, usually being (instruction-following) pairs where humans ask questions and the model replies. Supervised fine-tuning and parameter-efficient fine tuning methods are the most used ones, where only a small subset of the parameters is updated to avoid the LLM to forget its original knowledge.

2.8 RL AND MARL IN ROBOTICS

In the context of machine learning, RL is concerned with how an intelligent agent should take actions in a dynamic environment in order to maximize a reward signal. The agent must make decisions between trying new actions to learn more about the environment or using current knowledge of the environment to take the best action, balancing the exploration/exploitation trade.

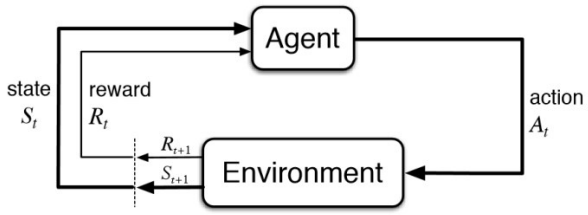

Figure 2.8: Basic RL worfklow schema: the agent iteratively maps states to actions by maximizing a cumulative reward signal through trial and error interaction.

Particularly their combination with deep neural networks, referred to as *deep reinforcement learning* (DRL) have shown remarkable capabilities in solving the most complex decision-making problems even with the high-dimensional observations in domains such as board games, video games, healthcare and recommendation systems [8].

2.8.1 Reinforcement Learning in robotic environments

Reinforcement learning offers to robotics a framework and set of tools for the design of sophisticated and hard-to-engineer behaviors.

The successes of combining deep neural networks for reinforcement learning policy training underscore the potential of DRL for controlling robotic systems with high-dimensional state or observation space and highly nonlinear dynamics to perform challenging tasks that conventional decision-making, planning, and control approaches (e.g., classical control, optimal control, sampling-based planning) cannot handle effectively. Furthermore, robots need to complete tasks in the physical world, which presents additional challenges. It is often inefficient and/or unsafe for the RL agents to collect trial-and-error samples directly in the physical world, and it is usually impossible to create an exact replica of the complex real world in simulation.

2.8.2 Multi Agent Reinforcement Learning

2.9 CURRENT SOTA VLMS

3. SOLUTION APPROACH

3.1 SOLUTION PROPOSAL

3.2 SYSTEM ARCHITECTURE

3.3 TOOLS USED

3.4 WORKFLOW

4. SOLUTION DEVELOPMENT

4.1 ISAACSIM ENVIRONMENT DEFINITION

4.1.1 Agents

4.1.2 Cameras

4.1.3 Physics

4.2 INDIVIDUAL COMPONENT OPERATION

5. EXPERIMENTATION AND EVOLUTION

5.1 FIRST SCENARIOS

5.2 LAST SCENARIO

5.3 SUCCESSFUL AND FAILURE TRIES

5.4 RESULTS OBTAINED IN EACH SCENARIO

5.5 RESULT DISCUSSION

6. CONCLUSIONS

6.1 CONCLUSIONS AND LESSONS LEARNED

6.2 LIMITATIONS FOUND

The main drawbacks I have found during the development and research process during the whole work have been mainly hardware-related as the specific software requirements necessary to carry on the development are severely demanding due to the fact that the majority of the processes are run and rendered in the GPU. Although I have had access to a good graphical processing unit, the other components of the my working device have fallen behind. Taking for instance the minimum requirements of the used framework *IsaacSim*, I found myself in the limits of the bare minimum, and some of my components such as the CPU were worse than the required. However, I have been able to carry on anyways although there have been a few moments where the I have really felt the bottleneck. When it comes those moments I have felt limited and slowed downed due to my equipment are the following:

- I have needed to split some of the working components of the pipeline process inside *IsaacSim* as the whole process would not fit in my GPU's VRAM. The simple fact of opening the framework already consumed close to a quarter of the graphic card's total capacity, and starting the server to listen and interact with the VLM calls alongside left little to no room for more processes in the background. The fact of working on the edge of the minimum hardware requirements also slowed the rendering of visual work and reduced the number of frames per second, nevertheless, this did not obstruct the development process.

- When it comes to making the most of the reinforcement learning training process, I have needed to tweak some of the predefined system configurations to maximize the performance and reduce any kind of lagging and overflow. Adapting the GPU capacity properties to maximize the overall functioning. If I were to work with better equipment, I would have liked to go for longer and deeper trainings, however, I am happy with the performance I obtained when tweaking the parameters I will have discussed in the corresponding section and other strategies such as parallelization.

- Not all limitations have been hardware related, as the official NVIDIA documentation websites were usually corrupted depending on the version. Leading to consulting external pages to solve the mismatching dependencies or installation processes for the frameworks used.

- After consulting the official documentation of the *IsaacLab* repository, I found that even the default reinforcement learning training examples and demonstrations did not work as intended independently of the version. I was not the only one with the same issues as other peoples raised the same issues in different forums, so I inspired in the solutions given by the users with the same problems.

6.3 FUTURE WORK

BIBLIOGRAPHY

[1] Multimodal learning. https://en.wikipedia.org/wiki/Multimodal_learning. Accessed: April 23, 2026.

[2] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale, 2021.

[3] Gravio Team. Vision-language models (VLM) vs visual question answering (VQA) in 2025? https://www.gravio.com/en-blog/vision-language-models-vlm-vs-visual-question-answering-vqa-in-2025, Mar 2025. Gravio Blog, Accessed: [Insert Date Here].

[4] Andrew Jaegle, Felix Gimeno, Andrew Brock, Andrew Zisserman, Oriol Vinyals, and Joao Carreira. Perceiver: General perception with iterative attention, 2021.

[5] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision, 2021.

[6] Nils Rethmeier and Isabelle Augenstein. A primer on contrastive pretraining in language processing: Methods, lessons learned and perspectives, 2021.

[7] Haochen Tan, Wei Shao, Han Wu, Ke Yang, and Linqi Song. A sentence is worth 128 pseudo tokens: A semantic-aware contrastive learning framework for sentence embeddings, 2022.

[8] Chen Tang, Ben Abbatematteo, Jiaheng Hu, Rohan Chandra, Roberto Martín-Martín, and Peter Stone. Deep reinforcement learning for robotics: A survey of real-world successes, 2024.

[9] M. Timothy. What is phrase grounding? https://blog.roboflow.com/what-is-phrase-grounding/, Nov 2024. Roboflow Blog, Accessed: [Insert Date Here].

[10] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023.

[11] Zirui Wang, Jiahui Yu, Adams Wei Yu, Zihang Dai, Yulia Tsvetkov, and Yuan Cao. Simvlm: Simple visual language model pretraining with weak supervision, 2022.

[12] Wikipedia. Cross-modal retrieval. https://en.wikipedia.org/wiki/Cross-modal_retrieval, 2026. Accessed: 2026-04-23.

[13] Wikipedia contributors. Bicubic interpolation — Wikipedia, the free encyclopedia. https://en.wikipedia.org/wiki/Bicubic_interpolation, 2024. [Online; accessed 19-April-2026].

[14] Wikipedia contributors. Contrastive language-image pre-training — Wikipedia, the free encyclopedia. https://en.wikipedia.org/wiki/Contrastive_Language-Image_Pre-training, 2024. [Online; accessed 19-April-2026].

[15] Wikipedia contributors. Knowledge distillation — Wikipedia, the free encyclopedia. https://en.wikipedia.org/wiki/Knowledge_distillation, 2024. [Online; accessed 20-April-2026].

[16] Wikipedia contributors. Manifold alignment — Wikipedia, the free encyclopedia, 2024. [Online; accessed 22-April-2026].

[17] Wikipedia contributors. Vision-language model — Wikipedia, the free encyclopedia, 2026.

7. APPENDIX

7.1 INSTALATION GUIDE

IsaacSim/Lab

7.2 RELEVANT CODE

7.3 PROMPTS USED

Prompts used for the VLM

7.4 RESULT TABLE

7.5 TENSORBOARD ANALYSIS